

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Utilização de bibliotecas de Ciência de Dados em Python

Introdução às bibliotecas de Ciência de Dados em Python

Manipulação e visualização de dados com
Pandas e Matplotlib

Código da aula: [DADOS]ANO1C1B4S32A2

**Utilização de
bibliotecas de
Ciência de Dados
em Python**

Mapa da unidade 7 Componente 1

semana

32

Você está aqui!

Introdução às bibliotecas de
Ciência de Dados em Python

**Utilização de
bibliotecas de
Ciência de Dados
em Python**

**Mapa da
unidade 7
Componente 1**

Você está aqui!

Introdução às bibliotecas de
Ciência de Dados em Python

Aula 2

Código da aula: [DADOS]ANO1C1B4S32A2

32



Objetivos da aula

- Introduzir e utilizar as principais bibliotecas de Ciência de Dados em Python.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da aula

50 minutos.



Competências técnicas

- Usar ferramentas de desenvolvimento de software como o Jupyter Notebook;
- Compreender e dominar as principais bibliotecas de Ciência de Dados em Python.



Competências socioemocionais

- Analisar informações de forma crítica e tomar decisões;
- Avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

Construindo
o **conceito**

Leitura de dados com Pandas

Pandas oferece funções para carregar dados de diversos formatos, como CSV, Excel, SQL, entre outros.

A função **read_csv** é frequentemente utilizada para importar dados de arquivos CSV para DataFrames, permitindo análises rápidas e eficientes.

- ▶ Vamos praticar a leitura a partir da criação de um arquivo .csv?

Construindo o conceito

Criando um arquivo CSV

```
import numpy as np

# Criando um DataFrame de exemplo com células vazias, NaN e dados repetidos
data = {
    'Nome': ['Ana', 'Bruno', 'Carlos', 'Daniela', 'Eduardo', 'Ana', 'Carlos', 'Fernanda', 'Gabriel', 'Helena',
            'Igor', 'Juliana', 'Lucas', 'Mariana', 'Nina', 'Oscar', 'Paula', 'Quintino', 'Rafaela', 'Sergio',
            'Tereza', 'Ulisses', 'Vanessa', 'Wagner', 'Ximena', 'Yara', 'Zeca', 'Ana', 'Bruno', 'Carlos'],
    'Idade': [23, 35, 45, 22, 34, 23, 45, np.nan, 30, 27,
            31, 40, np.nan, 25, 28, 33, 38, 29, 41, 50,
            32, 37, 29, 36, np.nan, 44, 39, 23, 35, np.nan],
    'Salario': [50000, 60000, 70000, 55000, 65000, np.nan, 70000, 58000, 62000, np.nan,
            50000, 59000, 72000, 54000, 53000, 58000, 61000, 63000, np.nan, 68000,
            52000, 60000, 57000, 64000, 69000, 55000, 66000, 50000, np.nan, 70000],
    'Departamento': ['Vendas', 'TI', 'RH', 'Financeiro', 'Marketing', 'Vendas', 'RH', 'TI', np.nan, 'Marketing',
            'TI', 'RH', 'Financeiro', 'Marketing', 'Vendas', 'TI', 'RH', 'Financeiro', 'Marketing', 'Vendas',
            'TI', 'RH', 'Financeiro', 'Marketing', 'Vendas', 'TI', 'RH', 'Financeiro', 'Marketing', 'RH']
}

df = pd.DataFrame(data)

# Salvando o DataFrame em um arquivo CSV
csv_path = 'dados_complexos.csv'
df.to_csv(csv_path, index=False)

csv_path
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Exemplo: leitura de dados com Pandas

Pandas fornece diversas funções para carregar dados de diferentes fontes, sendo a **função read_csv** uma das mais utilizadas.

Essa função permite importar dados de arquivos CSV para um DataFrame, uma estrutura de dados poderosa que facilita a manipulação e a análise de dados. Acompanhe o exemplo:

```
import pandas as pd

# Carregando dados de um arquivo CSV
df = pd.read_csv('dados_complexos.csv')
print(df.head())
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Limpeza e transformação de dados com Pandas

- ▶ A biblioteca Pandas facilita a limpeza e a transformação de dados, incluindo a manipulação de valores ausentes, duplicados e incorretos.
- ▶ Funções como **dropna**, **fillna** e **replace** são utilizadas para tratar esses problemas, garantindo a integridade dos dados. A seguir, confira na prática alguns exemplos de aplicação dessas funções.

Construindo o conceito

Exemplo: limpeza e transformação de dados com Pandas

A qualidade dos dados é fundamental para a análise precisa. Pandas oferece várias funções para limpeza e transformação de dados.

```
import pandas as pd

# Carregando dados de um arquivo CSV
df = pd.read_csv('dados_complexos.csv')

# Removendo linhas com valores ausentes
df_cleaned = df.dropna()
print("DataFrame após remover valores ausentes:")
print(df_cleaned)

# Preenchendo valores ausentes com a média das colunas numéricas
df_filled = df.fillna(df.select_dtypes(include=['float64', 'int64']).mean())
print("DataFrame após preencher valores ausentes com a média:")
print(df_filled)
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

- ▶ A função **dropna** remove linhas ou colunas com valores ausentes, enquanto **fillna** preenche esses valores com um valor especificado.

Construindo
o **conceito**

Operações com DataFrames

Pandas permite realizar operações complexas em DataFrames de maneira eficiente. A seleção de dados pode ser feita usando **loc** para acesso baseado em rótulos e **iloc** para acesso baseado em posições.



A **agregação** de dados é realizada com **groupby**, que permite agrupar dados por uma ou mais colunas e aplicar funções agregadas.

Construindo
o **conceito**

Exemplo: operações com DataFrames

Pandas permite realizar operações complexas em DataFrames de maneira eficiente.

A **seleção de dados** pode ser feita com **loc e iloc** para acesso baseado em rótulos e posições, respectivamente.

A **agregação de dados** pode ser realizada com **groupby**, que permite agrupar dados por uma ou mais colunas e aplicar funções agregadas. Vamos verificar como acontece na prática.

Construindo o conceito

Exemplo: operações com DataFrames

```
import pandas as pd

# Carregando dados de um arquivo CSV
df = pd.read_csv('dados_complexos.csv')

# Removendo linhas com valores ausentes
df_cleaned = df.dropna()
print("DataFrame após remover valores ausentes:")
print(df_cleaned)

# Preenchendo valores ausentes com a média das colunas numéricas
df_filled = df.fillna(df.select_dtypes(include=['float64', 'int64']).mean())
print("DataFrame após preencher valores ausentes com a média:")
print(df_filled)

# Agrupando dados por departamento e calculando a média de idade e salário
grouped_df = df_filled.groupby('Departamento')[['Idade', 'Salario']].mean()
print("Média de Idade e Salário por Departamento:")
print(grouped_df)
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Visualização de dados com Matplotlib

A visualização é um ponto fundamental para dados, certo?

Matplotlib permite a criação de gráficos de linhas, barras, dispersão, histogramas e muitos outros tipos de gráficos, facilitando a interpretação visual dos dados.

Essas características tornam Matplotlib uma biblioteca poderosa para criar visualizações de dados.

Construindo o conceito

Exemplo: visualização de dados com Matplotlib

Visualizar dados é crucial para identificar padrões e insights. Matplotlib permite criar uma variedade de gráficos, por exemplo, para criar um gráfico de linhas.

```
import pandas as pd
import matplotlib.pyplot as plt

# Carregando dados de um arquivo CSV
df = pd.read_csv('dados_complexos.csv')

# Removendo linhas com valores ausentes
df_cleaned = df.dropna()
print("DataFrame após remover valores ausentes:")
print(df_cleaned)

# Preenchendo valores ausentes com a média das colunas numéricas
df_filled = df.fillna(df.select_dtypes(include=['float64', 'int64']).mean())
print("DataFrame após preencher valores ausentes com a média:")
print(df_filled)

# Agrupando dados por departamento e calculando a média de idade e salário
grouped_df = df_filled.groupby('Departamento')[['Idade', 'Salario']].mean()
print("Média de Idade e Salário por Departamento:")
print(grouped_df)

# Ordenando por idade em ordem crescente
grouped_df_sorted = grouped_df.sort_values(by='Idade')
print("DataFrame ordenado por Idade:")
print(grouped_df_sorted)

# Criando um gráfico de linhas
plt.plot(grouped_df_sorted['Idade'], grouped_df_sorted['Salario'])
plt.xlabel('Idade Média')
plt.ylabel('Salário Médio')
plt.title('Gráfico de Linhas: Idade Média vs Salário Médio')
plt.show()
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Colocando
em **prática**

Trabalhando com arquivos CSV

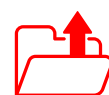
Agora é o momento de realizar as operações avançadas de manipulação de dados com o **arquivo 'dados_complexos.csv' que vocês criaram em aula.**



Durante a aula



Em dupla



Envie o arquivo .csv modificado com as suas respostas ao AVA



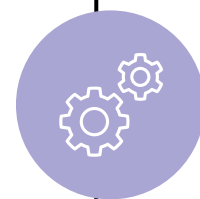
UTILIZAÇÃO DE ARQUIVO

Carregue o arquivo dados_complexos.csv no Pandas DataFrame.



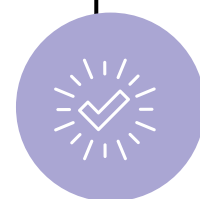
REMOÇÃO DE LINHAS

Remova as linhas com valores ausentes nas colunas Idade e Salário.



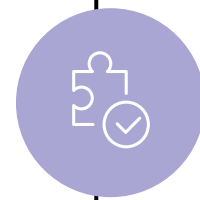
PREENCHIMENTO DE VALORES

Preencha os valores ausentes na coluna Departamento com o valor Desconhecido.



CRIAÇÃO DE NOVA COLUNA

Crie uma nova coluna chamada Faixa_Etaria que categorize as idades em: "Jovem" (até 30 anos), "Adulto" (31 a 45 anos) e "Sênior" (acima de 45 anos).

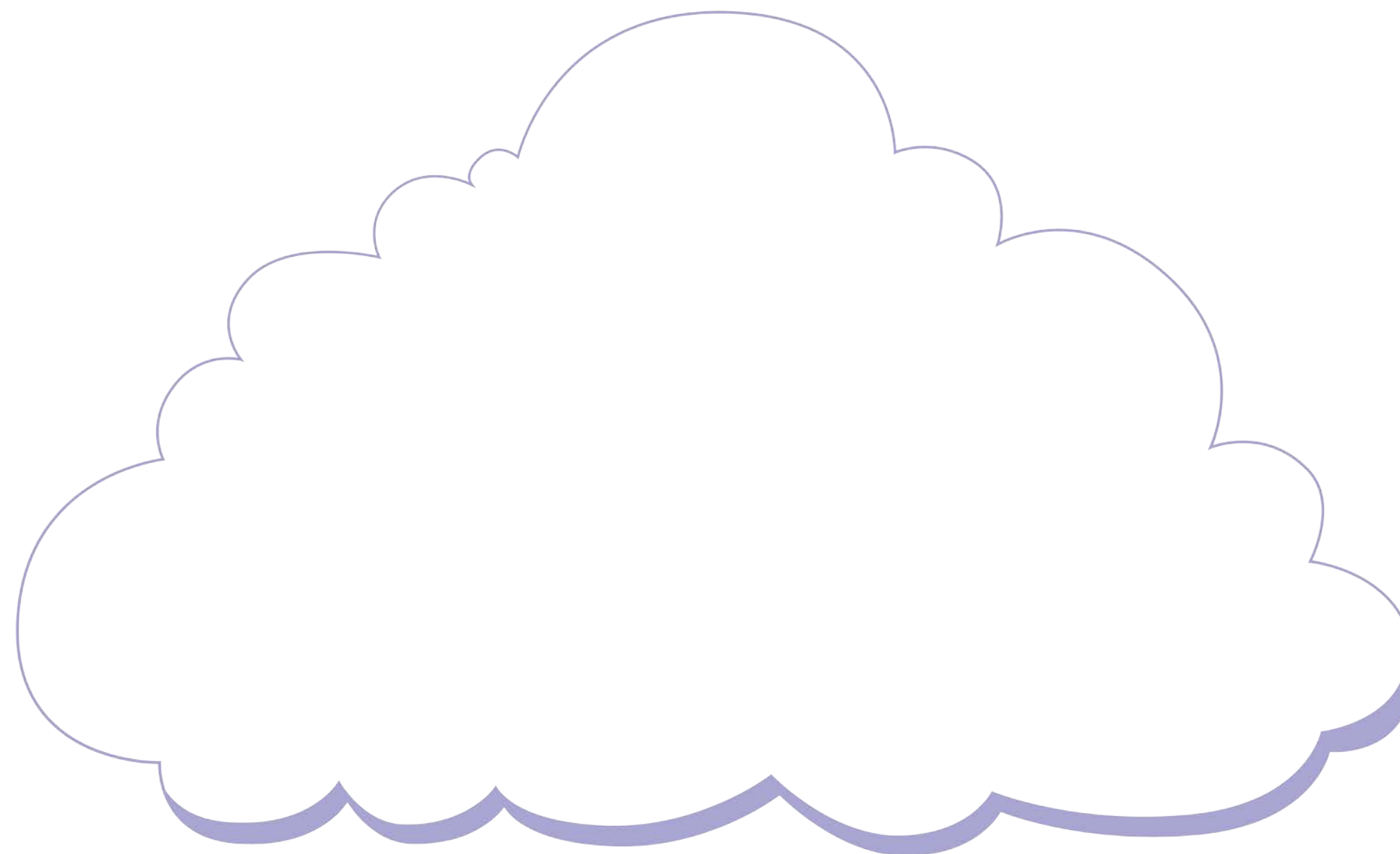


OPERAÇÃO PARA GRÁFICO

Crie um gráfico de barras que mostre a distribuição de salários por departamento, agrupadas por faixa etária.



Nuvem de palavras



© Getty Images

O que nós
aprendemos
hoje?



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** A aula abordou a manipulação e a visualização de dados com Pandas e Matplotlib, destacando a importância dessas bibliotecas na análise de dados.
- 2** Exploramos leitura, limpeza e transformação de dados em vários formatos, além de operações avançadas com DataFrames para preparação de análise.
- 3** Criamos visualizações com Matplotlib e Pandas, gerando gráficos diretamente de DataFrames para uma análise mais eficiente e intuitiva.

Saiba mais

Descubra como usar *seeds* para controlar aleatoriedade e explore as poderosas funções de agregação da biblioteca Numpy!

Acesse o link e saiba mais:

ALURA. *NumPy*: análise numérica eficiente com Python. Disponível em: <https://www.alura.com.br/curso-online-numpy-analise-numerica-eficiente-pythons>.

Acesso em: 20 ago. 2024.

Referências da aula

Identidade visual: imagens © Getty Images

JUPYTER. *Página inicial*, [s.d.]. Disponível em: <https://jupyter.org>. Acesso em: 20 ago. 2024.

MATPLOTLIB. *Página inicial*, [s.d.]. Disponível em: <https://matplotlib.org>. Acesso em: 20 ago. 2024.

NUMPY. *Página inicial*, [s.d.]. Disponível em: <https://numpy.org>. Acesso em: 20 ago. 2024.

PANDAS. *Página inicial*, [s.d.]. Disponível em: <https://pandas.pydata.org>. Acesso em: 20 ago. 2024.

SEABORN. *Página inicial*, [s.d.]. Disponível em: <https://seaborn.pydata.org>. Acesso em: 20 ago. 2024.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**